Exact finite lists for the girth–diameter middle-graph construction

Human Compression project: finite component of L013

We recover the finite assertions of Araujo-Pardo, Conder, García-Colín, Kiss and Leemans [1], using their middle-graph idea. The source is the complete eight-page arXiv v1, not an implicitly updated edition. For degrees three, four and five our calculation is exhaustive. For degree six we verify two examples and make no completeness assertion. All graph adjacencies, Boolean assignments, automorphism generators and identifying permutations are supplied as mathematical data. There is no new formalization or numerical length comparison.

1 The exact finite search space

A graph here is finite, simple and undirected. Fix $k \geq 3$, and write

$$V_k = \{(i, j) : 0 \leq i, j < k,\ i \neq j\}.$$

A possible middle edge joins two different rows and two different columns. For each such unordered pair e introduce a Boolean variable x_e. Impose the following conditions:

1. For each $v \in V_k$, $\sum_{e \ni v} x_e = k - 2$.
2. If a, b share a row or column and both va, vb are possible edges, then $x_{va} + x_{vb} \leq 1$.
3. For every possible triangle, its three variables have sum at most two.
4. For each possible four-cycle, its four variables have sum at most three.

The whole file `search.py` builds precisely these conditions. Duplicate forbidden edge sets are removed as sets, without removing any condition. It imposes no cyclic symmetry, lexicographic restriction, isomorphism pruning or connectivity condition on the middle graph.

Lemma 1. *The satisfying assignments are exactly the $(k-2)$-regular graphs on V_k with no triangles or four-cycles and with no path of length one or two between different vertices in a common row or column. Every k-regular graph of girth five, diameter four and order $k^2 + k + 2$ is obtained by extending at least one such assignment.*

Proof. The first assertion follows directly: possible edges already exclude equal rows or columns; the second condition excludes exactly the prohibited length-two paths; the last two conditions exclude exactly the short cycles.

For the second assertion, choose vertices r, c at distance four in the given graph. Their neighborhoods are disjoint, have no internal edges and have no edges between them. The $k(k-1)$ neighbors beyond $N(r)$ are distinct: a repeated neighbor gives a four-cycle through r. They avoid $r, c, N(r), N(c)$. The order hypothesis therefore makes them exactly the remaining set M. The same argument applies from c. Thus every vertex of M has exactly one neighbor in $N(r)$ and exactly one in $N(c)$, and has $k-2$ neighbors in M. Write the resulting rows and columns as R_i, C_j, each of size $k-1$. An intersection $R_i \cap C_j$ has size at most one, since two elements of it would give a four-cycle. Every row misses exactly one column, and every column misses exactly one row. Relabel the columns along this perfect matching of missing intersections. Then M is exactly V_k with the stated labels. An edge or a length-two path within a row or column gives a triangle or four-cycle in the full graph. The induced middle graph thus satisfies all four conditions. □

The extension adds row vertices r_i, column vertices c_j and two end vertices r, c, and edges rr_i, cc_j, $r_i(i,j)$ and $c_j(i,j)$. For every assignment in the finite lists, our separate literal adjacency checker verifies regularity, connectedness, exact girth five and exact diameter four in this extension. Consequently no sufficiency inference is hidden in the finite calculation. The lower bound $k^2 + k + 2$ follows from the same distinct-neighbor argument without assuming equality. Thus all reported examples attain the lower bound and are cages.

2 Enumeration and its trust boundary

The variable and constraint counts, completed labelled populations, and numbers of uncolored full-graph isomorphism classes are

k	variables	degree equations	forbidden sets	assignments	full classes
3	9	6	17	4	2
4	42	12	359	102	4
5	130	20	3815	240	7

These are exhaustive Boolean runs with OR-Tools CP-SAT 9.15.6755, one search worker and `enumerate_all_solutions=True`. Each run ended with `OPTIMAL`, rather than a time-limited feasible status. The complete assignments are retained, not only a count or one representative. A stopped or incomplete run does not certify the census. The required finite premise is this completed exact solver execution on the specified model; no kernel-checked SAT proof is claimed. Software version, commands and receipts are bound in the packet.

Pynauty 2.8.8.1 discovers convenient uncolored representatives and generators. It does not impose an unproved symmetry restriction on the search. Moreover, `finite_certificate.py` uses no SAT, nauty or graph-database import: it checks every listed assignment against the defining local conditions, constructs its full graph, and supplies an actual isomorphism to its claimed representative. It checks that different representatives are nonisomorphic by the complete elementary search described below. Thus a mistaken canonical key cannot silently merge two classes. The classified object is the full uncolored graph, not the rooted or row-colored middle graph.

3 Complete automorphism and identification certificates

Here is the exact algorithm in `direct_graphs.py`. To test an isomorphism $A \to B$, maintain paired ordered partitions of the two vertex sets. Repeatedly split each cell by the vector counting neighbors in every current cell. If corresponding cells have different signature multiplicities, reject the branch. If all cells are singletons, test the resulting permutation against every adjacency. Otherwise select the first smallest nonsingleton source cell, fix its first vertex a, and branch over every vertex b in the corresponding target cell, making a, b paired singleton cells. Refine and continue. For an isomorphism test one successful branch suffices; for an automorphism count every branch is explored.

Lemma 2. *The algorithm finds every graph isomorphism exactly once. In particular its full run with $A = B$ computes $|\operatorname{Aut}(A)|$.*

Proof. Every isomorphism respects the initial single-cell partitions. If it respects paired cells, it preserves every vector of neighbor counts, so it respects each refinement. A signature mismatch makes any such isomorphism impossible. At a branch, the image of a is exactly one of the enumerated vertices b; therefore exactly one child can contain a given isomorphism. Each branch strictly increases the number of singleton constraints, so the process terminates. At a leaf the unique candidate is explicitly checked. Induction over the search tree proves both exhaustiveness and absence of duplication. □

We also check each supplied generator as an actual adjacency-preserving permutation and close the generators under composition by finite breadth-first search. The resulting subgroup order equals the complete automorphism count in all fifteen graphs. These two sides prove full automorphism equality; there is no inference from a convenient subgroup alone.

k	order of full graph	automorphism orders, with multiplicity
3	14	$4, 12$
4	22	$1, 2, 4, 8$
5	32	$4, 4, 10, 10, 48, 64, 1920$
6	44	$40, 240$ (two witnesses only)

The degree-six inputs are the two literal graph6 strings in the data repository cited by the already available followup [2]. We independently decode and check them, including all automorphisms. We use the strings as witness data, not the followup's additional census assertion. Different automorphism orders already separate the two witnesses; the full nonisomorphism test also passes.

The source names six House of Graphs entries. Their actual API adjacencies were obtained and are bound by exact hashes. Saved permutations identify 1000 and 50487 with our degree-three classes (orders 12 and 4), and 50459, 49991, 49992 and 49993 with our degree-four classes (orders 1, 2, 4 and 8). This is an adjacency-level identification, not just an order comparison. The mathematical adjacency subset is exported; unrelated API metadata is not used. Source Figure 1 was also transcribed directly: its nineteen black edges plus its two red edges give 1000, while black plus green give 50487. `figure1_edges.json` specifies all fourteen vertices by their positions in the picture and all twenty-three drawn edges. Exact identifying permutations are included in the certificate report.

4 The source's chosen middle graphs

A middle graph depends on the chosen diametral pair. We test all unordered pairs at distance four in each representative and supply a specified pair realizing the following source choices. The degree-three middle is $3K_2$; its girth and diameter are recorded as absent rather than assigned a spurious finite value. For each degree-four class the chosen middle is C_{12}, verified by connectedness and degree two on twelve vertices. For degree five the chosen profiles are

$$2 \times (5, 4), \qquad 3 \times (5, 5), \qquad 2 \times (6, 4),$$

where a pair denotes (girth, diameter). The two order-four automorphism classes use $(6, 4)$, the two order-ten classes use $(5, 4)$, and the order-48, 64 and 1920 classes use $(5, 5)$. This selection matters: the order-48 class has six diametral pairs with middle profile $(5, 5)$ and two with $(6, 4)$. Thus the chosen profile is an existence statement for a specified pair, not an invariant of the full graph. All chosen pairs, all alternative profiles, and their original vertex sets are explicit in `finite_report.json`.

The finite component does not recover the infinite-family arguments, assert completeness in degree six, settle the source's future-work questions, or make a current record-holder claim. Those boundaries are separate from the exact finite conclusions proved here.

References

[1] G. Araujo-Pardo, M. Conder, N. García-Colín, Gy. Kiss and D. Leemans, *A note on girth-diameter cages*, arXiv:2401.15539v1, 28 January 2024, especially pp. 2–6.

[2] T. Van den Eede, *girthDiamGen*, data accompanying S. Cambie, J. Goedgebeur, J. Jooken and T. Van den Eede, *On the order-diameter ratio of girth-diameter cages*, `https://github.com/AGT-Kulak/girthDiamGen`; literal file `data/n44_k6_g5_d4.g6`, obtained 12 October 2026.

[3] K. Coolsaet, S. D'hondt and J. Goedgebeur, *House of Graphs 2.0*, Discrete Applied Mathematics 325 (2023), 97–107; exact six source-named adjacency records obtained 12 October 2026.